

Java Code Generation Method Based on UML Class Diagrams and Large Language Model

Linyun Cai

School of Information and
Software Engineering
University of Electronic Science
and Technology of China
Chengdu, China
3126483060@qq.com

Yuandan Deng

Intelligent Terminal Key
Laboratory of Sichuan Province
Yibin, China
denyuda@163.com

Weiqin Chen

University of Electronic Science
and Technology of China Yibin
Park
Yibin, China
2291027652@qq.com

Bo Chen*

School of Information and
Software Engineering
University of Electronic Science
and Technology of China
Chengdu, China
bochen@uestc.edu.cn

Abstract—We propose an automatic Java code generation method that integrates UML class diagrams with large language models. First, we employ DOM4J parsing and FreeMarker templates to transform complex, tool-exported UML XML into a unified and concise XML representation. We then construct *umltocode*, a task-specific dataset of aligned UML XML and Java pairs, and fine-tune CodeLlama to perform direct UML-to-Java generation. Finally, we evaluate generation quality using a structural consistency-oriented scoring strategy. Experimental results indicate that fine-tuning substantially improves both code structure fidelity and semantic accuracy, enabling fast and reliable synthesis of high-quality Java source code from class diagram specifications.

Keywords—code generation, UML class diagram, XML parsing, large model fine-tuning

I. INTRODUCTION

With the rapid growth of the software industry, traditional development workflows have become increasingly costly and time-consuming. As project scales expand, the burden of repetitive implementation and consistency checks also rises. To improve efficiency and shorten development cycles, developers have therefore turned to automated tools to assist the software engineering process. Among these tools, code generation has become a practical means of reducing manual effort and mitigating human errors. It also allows engineers to shift their attention from routine coding to more complex design and innovation tasks.

Model-driven code generation is an especially reliable paradigm. It separates business logic from technical implementation, describes system structure using abstract models such as the Unified Modeling Language (UML), and then produces code through model transformation. Among the various UML diagrams, class diagrams play a central role. They provide a static view of the system by specifying classes, attributes, operations, and inter-class relationships, which helps stakeholders understand object-oriented design more clearly [1]. However, the code generation functions embedded in existing UML tools usually produce only limited code skeletons. Functional completeness and relationship consistency often still require substantial manual work.

A feasible alternative is to extract model information from the XML files exported by UML tools and convert it into usable code. Yet these XML files typically contain redundant, tool-specific, or overly detailed metadata. This noise increases parsing complexity and reduces the usability of raw XML as model input. To address this problem, we employ the DOM4J parsing library together with FreeMarker templates to convert the exported XML into a concise and task-friendly format. We then fine-tune the large language model CodeLlama to directly generate Java code from these normalized XML files. Empirically, this approach can improve both generation accuracy and structural completeness, particularly for complex diagrams involving interfaces, static elements, and abstract classes.

The main contributions of this work are threefold:

- 1) We design a DOM4J- and FreeMarker-based pipeline that converts complex UML class diagrams into a unified and concise XML format suitable for large language models.
- 2) We construct a task-specific dataset, *umltocode*, consisting of aligned UML XML and Java code pairs that cover commonly used class diagram constructs.
- 3) We fine-tune CodeLlama-7B on this dataset and propose a structural consistency-based evaluation strategy, demonstrating improved UML-faithful Java generation compared with strong baselines.

II. RELATED WORK

Generating source code from models has long been an important topic in software engineering. In particular, UML class diagrams are widely adopted because they offer an intuitive representation of system architecture and object-oriented design. Over the years, researchers have explored multiple directions to automate the conversion from UML to code. These directions include rule-based systems, model-driven development (MDD), and more recently, large language model (LLM)-based approaches.

Early studies mainly focused on rule-based systems. Such methods rely on predefined transformation rules that map model constructs to programming-language constructs. Their

This work was supported by Sichuan Province Scientific and Technological Achievements Transfer and Transformation Demonstration Project (No. 2024ZHC0029) and Yibin City Twin City School Agreement Special Research Fund Science and Technology Project (No. YBSCXY2024010002).

advantages lie in simplicity and interpretability. However, their limitations are equally clear. Hard-coded rules are difficult to maintain and often fail to generalize to complex patterns or evolving requirements. For example, a business rule-driven approach and corresponding tool were proposed to automatically convert business rules into source code [2]. Code Swarm (CodS) further uses the SSwarm method to derive transformation rule sets and generate implementation code from system design models [3].

MDD emphasizes the use of abstract models instead of direct coding. Tools and frameworks in this paradigm typically define conversion rules and apply model transformation engines to generate target code. By incorporating object-oriented and modular concepts, prior work has proposed conversion rules between UML diagrams and Java code to enable automatic generation [4]. Acceleo, implemented in the Eclipse ecosystem, allows developers to define model-to-text transformation rules in a structured manner. UJECTOR was introduced to convert UML models into executable code while maintaining model-code consistency [5]. Other studies used XML as an intermediate representation to generate Java code from class and sequence diagrams [6]. JC_Gen extracts information from XMI patterns of activity and sequence diagrams exported by BoUML to automate code generation [7]. A broader analysis of UML↔code bidirectional conversion also highlighted implementation challenges and costs, providing valuable guidance for practitioners [8]. Despite these strengths, MDD-based approaches still require substantial rule engineering and often remain limited in flexibility.

In recent years, LLMs have emerged as promising general-purpose code generators. Researchers have started to examine whether these models can support UML-to-code tasks. A recent investigation evaluated GPT-4-Vision for converting UML class diagrams into actionable Java classes [9]. The model performed well on single-class diagrams but showed limitations on multi-class inputs. GraphCodeBERT incorporates graph-structured learning to capture structured information in code-related tasks [10]. CodeT5 improves generation quality through a unified pre-trained encoder-decoder framework [11]. Codex, fine-tuned on large-scale GitHub code, demonstrated that repeated sampling can produce effective solutions for difficult prompts [12]. LLM-based code generation has also been explored beyond software. For example, fine-tuning on domain-specific datasets improved Verilog generation performance in hardware design tasks [13]. Other studies proposed enhanced frameworks that combine LLM program generation with routing mechanisms [14] or endow models with requirement clarification abilities, such as ClarifyGPT [15].

Reliability and robustness remain important concerns. The RobustAPI dataset was introduced to evaluate LLM-generated code quality and revealed potential API misuse and downstream risks [16]. A systematic review further indicated that code generation by AI models may introduce security vulnerabilities in practical scenarios [17]. Overall, existing research suggests that while LLMs are strong at general coding tasks, they may still struggle with strict, domain-specific structural constraints. This gap motivates targeted dataset construction and task-specific fine-tuning for UML-to-Java generation.

III. METHOD

A. DOM4J Parsing

DOM4J is an open-source Java library used for parsing, modifying, and generating XML documents. An XML file defines data structures through hierarchical tags. For UML class diagram conversion, it is crucial to reliably extract nested elements such as class definitions, attributes, operations, and inter-class relationships. DOM4J supports XPath and provides flexible APIs for traversing nodes, which makes it suitable for processing complex, tool-exported XML files. The process of parsing class diagram XML files using DOM4J is shown in Fig. 1.

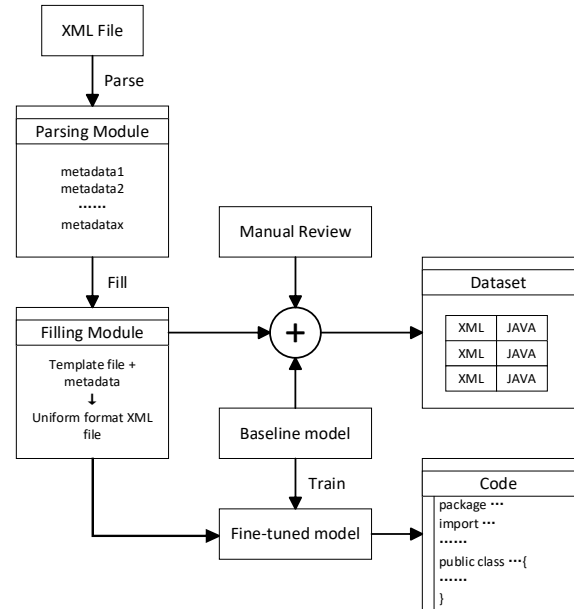


Fig. 1. Parse the class diagram XML file using DOM4J.

In our approach, DOM4J parses the original UML XML and extracts key information including class names, attributes, methods, and relationships. This information is then organized into a well-defined intermediate data model. The data model serves as the structured input for the subsequent template-based normalization stage.

B. FreeMarker Template

FreeMarker is a Java template engine that generates output text (e.g., HTML, XML, or source code) by combining predefined templates with dynamic data. Its script-like syntax is lightweight and easy to maintain. This makes it a convenient tool for formatting heterogeneous XML content into a unified representation.

In this study, we design FreeMarker templates that preserve essential class diagram semantics while removing redundant metadata. The templates cover classes, attributes, operations, and common relationship types. Therefore, any XML file exported by mainstream UML tools can be transformed into a consistent, compact XML format. A simplified template example is shown in Table I.

The transformation procedure is straightforward. First, the DOM4J-based parser extracts metadata from the original XML

and stores it in structured collections. For instance, for each attribute we record attribute.type and attribute.name. Next, the FreeMarker filling module injects these collections into the template. This step produces the simplified class diagram XML file. By combining DOM4J parsing and FreeMarker-based normalization, we obtain a unified XML format that is more suitable as input to large language models. The overall workflow is illustrated in Fig. 2. We collect UML class diagram XML files, parse them to obtain metadata, and then generate normalized XML files through template filling. An untuned large language model produces initial Java code from these normalized inputs. After manual review and correction, we construct an XML–Java mapping dataset. Finally, we fine-tune the base model on this dataset. When new class diagrams need to be converted, the same parsing and filling steps are applied, and the fine-tuned model outputs Java code that better matches the diagram specifications.

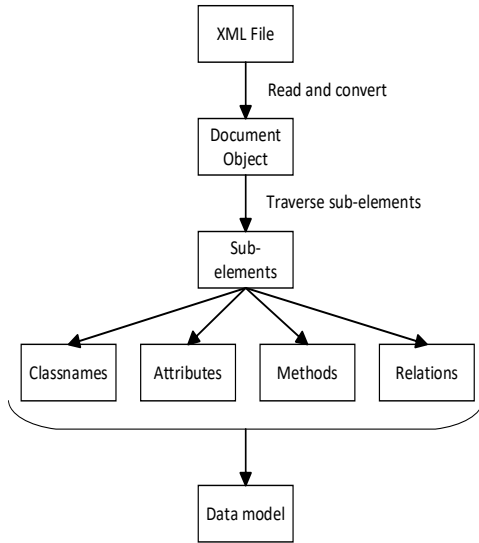


Fig. 2. Method schematic diagram.

TABLE I. FREEMARKER TEMPLATE

FreeMarker Template Example
<pre> package \${package}; public class \${className} { <#list attributes as attribute> private \${attribute.type} \${attribute.name}; </#list> <#list attributes as attribute> public \${attribute.type} get\${attribute.name}() { return \${attribute.name}; } public void set\${attribute.name}(\${attribute.type} \${attribute.name}) { this.\${attribute.name} = \${attribute.name}; } </#list> <#list operations as operation> public \${operation.returnType} \${operation.name}() { // TODO: implement \${operation.name} } </#list> } </pre>

IV. EXPERIMENT

A. Dataset Construction and Quality Control

The dataset construction is based on simplified XML class diagram files. We build a dataset named umltocode to fine-tune a large language model so that it can generate Java code conforming to class diagram specifications from the unified XML format. In fact, strong LLMs such as GPT-4 and Llama3-70B can already generate relatively complete Java code from the normalized XML. Nevertheless, even with detailed instructions, their outputs may still contain defects in UML-specific aspects such as interfaces, static elements, and abstract classes. These errors become more frequent as the class diagrams grow more complex.

To reduce such inconsistencies, we construct a task-specific dataset. Part of the XML content comes from collected and manually produced class diagrams, which are transformed into the unified format using the proposed parsing and templating pipeline. For the code side, AI-generated Java is used as an initial draft. We then manually revise the code that does not match the original class diagram or violates UML semantics. To mitigate subjective bias, we develop detailed annotation guidelines aligned with UML class diagram definitions. For a subset of instances, two annotators independently verify the generated code. Disagreements are resolved through discussion, achieving 95% consistency within the double-annotated samples. In addition, we implement automatic consistency checks to ensure that class names, attributes, methods, and inter-class relationships in the Java code match the information specified in the XML files.

Each instance in umltocode contains three fields: “instruction”, “input”, and “output”. They correspond to the prompt, the normalized class diagram XML, and the final Java code, respectively. Using the above process, we construct 2230 data instances that cover major class diagram constructs. The dataset is organized in JSON format to facilitate training. An example is shown in Fig. 3.

```

{
  "instruction": "Please generate the corresponding Java class code based on the following UML class diagram file in XML format. Requirements: It should fully include the attributes and methods of the class, and try to write the specific implementation based on the method name: ",
  "input": "<Class name=\"Animal\">\n  <Attributes>\n    <Attribute name=\"species\" type=\"String\"/>\n    <Attribute name=\"age\" type=\"int\"/>\n  </Attributes>\n  <Operations>\n    <Operation name=\"eat\" returnType=\"void\">\n      <Parameter name=\"food\" type=\"String\"/>\n    </Operation>\n    <Operation name=\"makeSound\" returnType=\"void\"/>\n  </Operations>\n</Class>",
  "output": "public class Animal {\n    private String species;\n    private int age;\n    public Animal(String species, int age) {\n        this.species = species;\n        this.age = age;\n    }\n    public String getSpecies() {\n        return species;\n    }\n    public void setSpecies(String species) {\n        this.species = species;\n    }\n    public int getAge() {\n        return age;\n    }\n    public void setAge(int age) {\n        this.age = age;\n    }\n    public void eat(String food) {\n        System.out.println(species + \" is eating \" + food);\n    }\n    public void makeSound() {\n        System.out.println(species + \" is making a sound\");\n    }\n}"
}

```

Fig. 3. Dataset format.

Before training, we collect a large number of UML XML files and apply the normalization pipeline. The resulting training and test sets are then used for fine-tuning and evaluation. The project is deployed in VS Code and consists of two modules:

XML parsing and model training. The former is implemented in Java with FreeMarker and managed by Maven. The latter is implemented using PyTorch and runs on a workstation equipped with an NVIDIA GeForce RTX 4060 GPU. We select CodeLlama-7B as the base model and adopt the Unsloth training framework. The AdamW optimizer and a linear learning rate scheduler are used for supervised fine-tuning. During training, hyperparameters such as learning rate and batch size are adjusted, and the model with the best validation performance is selected for testing.

B. Evaluation Metrics

To evaluate performance before and after fine-tuning, we design an evaluation strategy that reflects the relative importance of different code elements. Specifically, class existence, attributes and operations, method implementations, modifiers, and inter-class relationships are assigned weights of 1, 1.5, 1, 0.5, and 1, respectively. These weights are intended to capture the impact of each element on preserving UML semantics in the generated code.

We also experimented with alternative weighting schemes. The relative ranking of the compared models remains stable under reasonable changes, which suggests that the conclusions are not sensitive to the exact weight configuration. For each class diagram in the test set, the maximum score is computed in advance based on this strategy. We then score the Java code generated by each model and aggregate the results.

C. Baselines and Comparative Results

The experimental results are reported in Table II. In addition to the CodeLlama-7B base model, we also evaluate recognized strong LLMs such as GPT-4 and Llama3-70B. During evaluation, test diagrams are divided into single-class and multi-class categories. Single-class diagrams contain only one class with its attributes and operations. In contrast, multi-class diagrams include common relationship and specification elements and therefore represent more complex inputs.

TABLE II. PERFORMANCE COMPARISON OF THE MODEL BEFORE AND AFTER FINE-TUNING

Class Type	Test Class	Max Score	Base line	GPT -4	Llama3 -70b	Finetuned
Single-class	worker	9	9	9	9	9
	bookstore	17	13	16	16	17
	bag	28	22	24	26	28
Multi-class	displayer	14	11	12	12	14
	crops	21.5	17	19	19	21.5
	weapons	35	29	31	30	35
	army	15	10	12	13	15
	wing	14	9	10	10	14
	fish	18.5	12	13	13.5	18.5
	observe	24	14	18.5	21	24
	galaxy	11	7	11	8	11
	museum	16	12	15	13	16
	operable	55	37	49.5	46	55
	transport	61	41	45.5	44	59

The “Maximum Score” column in Table II indicates the upper bound for each test diagram under the proposed evaluation strategy. The subsequent columns report the actual scores of the Java code generated by the baseline model and the fine-tuned model. The results show a clear improvement after fine-tuning. In most cases, the fine-tuned scores closely approach the maximum scores, indicating strong structural fidelity to the original diagrams.

As diagram complexity increases, the fine-tuned model maintains higher normalized scores. This suggests improved robustness on complex multi-class specifications. The corresponding line graph is shown in Fig. 4. We additionally adopt CodeBLEU with Java as the target language to provide a general-purpose evaluation of code generation quality. CodeBLEU is widely used in code generation tasks because it measures similarity from multiple dimensions. The fine-tuned model improves both the structural consistency score and CodeBLEU, indicating that the observed gains are not an artifact of our task-specific metric. The CodeBLEU scores of each model on the same test set are shown in Table III.

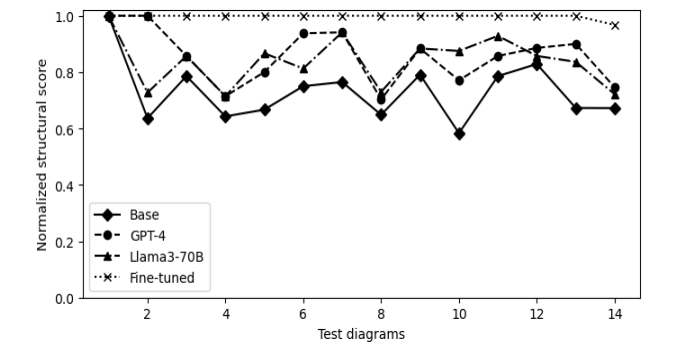


Fig. 4. The performance of the four models on the same test set (each horizontal coordinate represents a class in the test set, arranged from left to right in order of complexity).

	CodeB LEU	ngram	Weighted ngram	Syntax match	Dataflow match
Baseline	0.203	0.115	0.249	0.357	0.093
GPT-4	0.703	0.696	0.724	0.747	0.646
Llama3-70b	0.650	0.621	0.634	0.679	0.665
Finetuned	0.876	0.863	0.875	0.891	0.875

In contrast, the non-fine-tuned model occasionally omits key diagram elements or fails to satisfy relationship constraints. This outcome may be attributed to the ambiguous interpretation of task requirements by general-purpose models. In other words, what off-the-shelf LLMs lack in this scenario is not the ability to generate syntactically valid code, but the ability to precisely follow domain-specific structural specifications. This is exactly why constructing a focused dataset and performing targeted fine-tuning are essential for reliable UML-to-Java generation.

V. CONCLUSION

This paper presents a pipeline that transforms complex UML class diagram XML files into a unified format and constructs an XML–Java mapping dataset for fine-tuning large language models. Experimental results show that the fine-tuned model

significantly outperforms the original CodeLlama-7B. It also achieves higher structural consistency than strong general-purpose models such as GPT-4 and Llama3-70B on the same test set. Both the structure and accuracy of the generated Java code are improved, and the outputs better satisfy UML class diagram specifications.

Beyond the UML-to-Java scenario, our approach illustrates how domain-specific model normalization and lightweight fine-tuning can bridge the gap between general-purpose LLMs and strict requirements in model-driven engineering. The same idea is applicable to other UML diagram types, such as state or activity diagrams, and can be extended to other implementation languages. This provides a promising direction for integrating LLMs into practical, industrial-grade code generation workflows.

REFERENCES

- [1] M. Ozkaya, "Are the UML modelling tools powerful enough for practitioners? A literature review," *IET Software*, vol. 13, no. 5, pp. 338–354, 2019.
- [2] A. Kopp and D. Orlovskyi, "An approach and a software tool for automatic source code generation driven by business rules," in *Proc. COLINS*, 2022, pp. 338–353.
- [3] H. Mahmood, A. A. Jilani, and A. Rauf, "Code Swarm: a code generation tool based on the automatic derivation of transformation rule set," *Int. J. Software Eng. Appl. (IJSEA)*, vol. 14, no. 6, 2023.
- [4] S. Liu and X. Gao, "Research and implementation of Java code automatic generator based on UML," *Computer Life*, vol. 12, no. 2, pp. 37–42, 2024.
- [5] M. Usman, A. Nadeem, and T. Kim, "UJECTOR: a tool for executable code generation from UML models," in *Proc. Advanced Software Engineering and Its Applications*, 2008, pp. 165–170.
- [6] P. Kluisitrakul and Y. Limpiyakorn, "Generation of Java code from UML sequence and class diagrams," in *Proc. Int. Conf. Information Science and Applications (ICISA)*, 2016, pp. 1117–1125.
- [7] A. D. Durai, M. Ganesh, R. M. Mathew, and D. K. Anguraj, "A novel approach with an extensive case study and experiment for automatic code generation from the XMI schema of UML models," *J. Supercomput.*, pp. 1–23, 2022.
- [8] D. C. Apostol, P. D. Rusovan, and M. Marcu, "UML to code, and code to UML, a view inside implementation challenges and cost," in *Proc. 26th Int. Conf. System Theory, Control and Computing (ICSTCC)*, 2022, pp. 140–145.
- [9] G. Antal, R. Vozár, and R. Ferenc, "Toward a new era of rapid development: assessing GPT-4-Vision's capabilities in UML-based code generation," in *Proc. 1st Int. Workshop on Large Language Models for Code*, 2024.
- [10] D. Guo et al., "GraphCodeBERT: Pre-training code representations with data flow," unpublished, arXiv:2009.08366, 2020.
- [11] Y. Wang et al., "CodeT5: identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," unpublished, arXiv:2109.00859, 2021.
- [12] M. Chen, "Evaluating large language models trained on code," unpublished, arXiv:2107.03374, 2021.
- [13] S. Thakur et al., "VeriGen: a large language model for Verilog code generation," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 3, pp. 1–31, 2024.
- [14] X. Li and T. Döhmen, "Towards efficient data wrangling with LLMs using code generation," in *Proc. Eighth Workshop on Data Management for End-to-End Machine Learning*, 2024, pp. 62–66.
- [15] F. Mu et al., "ClarifyGPT: empowering LLM-based code generation with intention clarification," unpublished, arXiv:2310.10996, 2023.
- [16] L. Zhong and Z. Wang, "Can LLM replace Stack Overflow? A study on robustness and reliability of large language model code generation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 19, pp. 21841–21849, 2024.
- [17] C. Negri-Ribalta, R. Geraud-Stewart, A. Sergeeva, and G. Lenzini, "A systematic literature review on the impact of AI models on the security of code generation," *Frontiers Big Data*, vol. 7, Art. no. 1386720, 2024.